PANDUAN PERSIAPAN PESERTA

Persiapan Laptop Peserta

Training Arsitektur Microservices dan Cloud dengan Golang

DISIAPKAN UNTUK

Peserta In-house Training

Training	**26 – 29 Oktober 2026**
Batas persiapan	**Jumat, 23 Oktober 2026**
Perkiraan waktu	**1 – 2 jam, *download* ±5 GB**
Sistem operasi	**Windows 10/11 (WSL2), macOS, Linux**

PT ArtiVisi Intermedia
endy@artivisi.com · artivisi.com

19 Oktober 2026

1 Ringkasan

Setiap peserta menjalankan seluruh lab di laptop masing-masing. Panduan ini menyiapkan laptop tersebut dalam empat langkah:

1. Memastikan spesifikasi laptop dan hak akses memenuhi syarat (bagian 2).
2. Memastikan jaringan kantor membuka akses ke alamat yang dibutuhkan (bagian 3).
3. Memasang *software* sesuai sistem operasi (bagian 5, bagian 6, bagian 7, bagian 8).
4. Menjalankan skrip pemeriksaan, lalu mengirim *file* log hasilnya ke PIC (bagian 9).

Seluruh langkah selesai paling lambat **Jumat, 23 Oktober 2026**. Pada tanggal tersebut instruktur dan PIC melakukan pemeriksaan jarak jauh terhadap hasil skrip dari setiap laptop. Laptop yang belum lulus pemeriksaan masih dapat diperbaiki sebelum hari pertama training.

> Pertanyaan seputar persiapan laptop disampaikan melalui PIC, atau langsung ke instruktur: Endy Muhardin, `endy@artivisi.com`.

2 Spesifikasi dan Hak Akses

Komponen	Syarat
Prosesor	64-bit, x86_64 (Intel/AMD) atau Apple Silicon
RAM	Minimal 16 GB. Docker mendapat jatah minimal 8 GB dari jumlah tersebut
Ruang disk	Minimal 25 GB kosong
Sistem operasi	Windows 10 versi 22H2 atau Windows 11 dengan WSL2, macOS 13 atau lebih baru, atau Linux 64-bit (Ubuntu 22.04/24.04 sudah diuji)
Hak akses	Hak administrator untuk memasang *software* dan menjalankan Docker
Virtualisasi	Windows: fitur *virtualization* (Intel VT-x / AMD-V) aktif di BIOS. Kebijakan laptop kantor sering menonaktifkan fitur ini, sehingga perlu diperiksa sebelum memasang Docker Desktop

> Bila laptop kantor tidak dapat memenuhi hak akses atau virtualisasi, siapkan laptop khusus untuk training dan sampaikan ke PIC sebelum 23 Oktober.

3 Akses Jaringan

Jaringan tempat laptop digunakan, baik kantor maupun Wi-Fi ruang training, harus membuka akses keluar ke alamat berikut.

Alamat	Port	Kegunaan
`git.training.artivisi.id`	22, 443	Git server dan *container registry* training. Port 22 (SSH) wajib terbuka karena *clone*, *push*, dan login ke server *deployment* menggunakan SSH

`*.training.artivisi.id`	22, 443	Server *deployment* dan aplikasi peserta (Day 4)
`proxy.golang.org`, `sum.golang.org`	443	*Download* dan verifikasi Go module
`go.dev`, `dl.google.com`	443	Installer Go
`registry-1.docker.io`, `auth.docker.io`, `production.cloudflare.docker.com`	443	Docker Hub
`ghcr.io`, `pkg-containers.githubusercontent.com`	443	Image pendukung k3d
`github.com`, `raw.githubusercontent.com`, `objects.githubusercontent.com`, `release-assets.githubusercontent.com`	443	Repositori materi training, skrip pemeriksaan, *download* protoc, k3d, dan dependensi lain
`dl.k8s.io`	443	*Download* kubectl (Linux dan WSL2)

Jaringan kantor yang mewajibkan HTTP *proxy* membutuhkan dua pengaturan tambahan:

- **Terminal.** Tambahkan baris berikut ke `~/.bashrc` (Linux, WSL2) atau `~/.zshrc` (macOS), dengan alamat *proxy* dari tim jaringan:

```
export HTTPS_PROXY=http://proxy.kantor:8080
export HTTP_PROXY=http://proxy.kantor:8080
export NO_PROXY=localhost,127.0.0.1,.local
```

- **Docker Desktop.** Menu *Settings* → *Resources* → *Proxies*, isi alamat *proxy* yang sama.

SSH ke port 22 tidak melewati HTTP *proxy*. Akses langsung ke `git.training.artivisi.id` port 22 perlu dibuka oleh tim jaringan.

4 Versi Software

Software	Versi	Keterangan
Go	1.27.x	Minimal 1.26
protoc	36.2	*Protocol Buffers compiler*
protoc-gen-go	1.36.12	Plugin protoc untuk Go
protoc-gen-go-grpc	1.6.2	Plugin protoc untuk gRPC
Docker	Versi terbaru	Docker Desktop (Windows, macOS) atau Docker Engine dengan Compose *plugin* (Linux)
k3d	5.9.0	Cluster Kubernetes di atas Docker
kubectl	1.37.x	Minimal 1.35
Git, Make, curl	Bawaan sistem	Sudah tersedia di macOS dan Ubuntu, atau dipasang pada langkah instalasi

5 Instalasi di Windows

Di Windows seluruh lab berjalan di dalam Ubuntu pada WSL2. Perintah lab ditulis untuk *shell* Linux, sehingga peserta Windows dan Linux menjalankan perintah yang sama.

5.1 Aktifkan WSL2 dan Ubuntu

Buka PowerShell sebagai *Administrator*, lalu jalankan:

```
wsl --install -d Ubuntu-24.04
```

Restart laptop. Setelah restart, jendela Ubuntu terbuka dan meminta *username* serta *password* baru untuk Ubuntu.

5.2 Batasi Memori WSL2

WSL2 secara bawaan mengambil setengah RAM laptop. Buat *file* `%UserProfile%\.wslconfig` dengan Notepad:

```
[wsl2]
memory=10GB
```

Simpan, lalu jalankan `wsl --shutdown` di PowerShell agar pengaturan berlaku.

5.3 Pasang Docker Desktop

1. *Download* Docker Desktop dari `docs.docker.com/desktop/setup/install/windows-install` dan pasang dengan opsi *Use WSL 2 instead of Hyper-V*.
2. Buka Docker Desktop, menu *Settings* → *Resources* → *WSL integration*, aktifkan **Ubuntu-24.04**, lalu klik *Apply & restart*.
3. Buka terminal Ubuntu dan jalankan `docker version`. Keluaran yang memuat bagian `Server` menandakan Docker dapat diakses dari Ubuntu.

5.4 Lanjutkan di Ubuntu

Langkah berikutnya dijalankan di terminal Ubuntu: bagian 7 (lewati bagian Docker Engine, karena Docker Desktop sudah terpasang), lalu bagian 8.

Editor yang disarankan adalah Visual Studio Code dengan ekstensi **WSL** dan **Go**. Jalankan `code .` dari terminal Ubuntu untuk membuka folder proyek di dalam WSL2.

6 Instalasi di macOS

1. Pasang Command Line Tools (berisi Git dan Make):

   ```
   xcode-select --install
   ```

2. Pasang Homebrew dari `brew.sh` bila belum tersedia.
3. Pasang Go, protoc, k3d, dan kubectl:

   ```
   brew install go protobuf k3d kubernetes-cli
   ```

4. Tambahkan folder *binary* Go ke `PATH`:

   ```
   echo 'export PATH="$PATH:$HOME/go/bin"' >> ~/.zshrc
   source ~/.zshrc
   ```

5. Pasang Docker Desktop dari `docs.docker.com/desktop/setup/install/mac-install`, sesuai jenis prosesor (Apple Silicon atau Intel).

6. Di Docker Desktop, menu *Settings* → *Resources*, atur *Memory* minimal 8 GB dan *Virtual disk limit* minimal 40 GB, lalu klik *Apply & restart*.

Lanjutkan ke bagian 8.

7 Instalasi di Linux dan Ubuntu WSL2

Perintah di bagian ini ditulis untuk prosesor x86_64. Untuk prosesor ARM, ganti `amd64` dengan `arm64` dan `x86_64` dengan `aarch_64`.

7.1 Paket Dasar

```
sudo apt update
sudo apt install -y curl git make unzip
```

7.2 Go

```
curl -LO https://go.dev/dl/go1.27.2.linux-amd64.tar.gz
sudo rm -rf /usr/local/go
sudo tar -C /usr/local -xzf go1.27.2.linux-amd64.tar.gz
echo 'export PATH="$PATH:/usr/local/go/bin:$HOME/go/bin"' >> ~/.bashrc
source ~/.bashrc
go version
```

7.3 protoc

```
curl -LO https://github.com/protocolbuffers/protobuf/releases/download/v36.2/protoc-36.
2-linux-x86_64.zip
sudo unzip -o protoc-36.2-linux-x86_64.zip -d /usr/local bin/protoc 'include/*'
protoc --version
```

7.4 k3d dan kubectl

```
curl -Lo k3d https://github.com/k3d-io/k3d/releases/download/v5.9.0/k3d-linux-amd64
sudo install -m 0755 k3d /usr/local/bin/k3d
curl -LO https://dl.k8s.io/release/v1.37.1/bin/linux/amd64/kubectl
sudo install -m 0755 kubectl /usr/local/bin/kubectl
k3d version
kubectl version --client
```

7.5 Docker Engine (Linux saja)

Peserta WSL2 melewati bagian ini. Di Linux, pasang Docker Engine beserta Compose *plugin* mengikuti `docs.docker.com/engine/install` sesuai distribusi, lalu izinkan *user* menjalankan Docker tanpa `sudo`:

```
sudo usermod -aG docker $USER
```

Logout lalu login kembali agar keanggotaan grup `docker` berlaku.

8 Plugin protoc (Semua Sistem Operasi)

```
go install google.golang.org/protobuf/cmd/protoc-gen-go@v1.36.12
go install google.golang.org/grpc/cmd/protoc-gen-go-grpc@v1.6.2
protoc-gen-go --version
protoc-gen-go-grpc --version
```

Perintah ini sekaligus menguji akses ke `proxy.golang.org`. Kegagalan *download* di langkah ini umumnya disebabkan *proxy* atau *firewall* kantor (bagian 3).

Editor: Visual Studio Code dengan ekstensi **Go** (dari Go Team at Google), atau GoLand.

9 Pemeriksaan Kesiapan

Skrip pemeriksaan memeriksa versi *software*, memori Docker, ruang disk, dan akses jaringan, melakukan *download* seluruh image Docker yang digunakan selama training, lalu menguji `docker compose up` dan `k3d cluster create`. Jalankan dari terminal macOS, terminal Ubuntu WSL2, atau terminal Linux, dengan Docker dalam keadaan berjalan:

```
curl -fsSL https://raw.githubusercontent.com/artivisi/training-microservices-2026-01/
main/setup-guide/cek-persiapan.sh | bash
```

Pemeriksaan pertama memakan waktu 10 – 20 menit karena *download* image ±4 GB. Contoh keluaran laptop yang siap:

```
== Perangkat lunak
[OK]    Go 1.27.2
[OK]    protoc 36.2
...
== Akses jaringan
[OK]    Git server training (SSH port 22)
...
== Cluster Kubernetes lokal (k3d)
[OK]    k3d cluster create, node Ready

HASIL: SEMUA PEMERIKSAAN LULUS
```

Hasil pemeriksaan tersimpan di `~/cek-persiapan.log`. Kirim *file* tersebut ke PIC paling lambat **Jumat, 23 Oktober 2026**, termasuk bila masih ada pemeriksaan yang gagal.

9.1 Penanganan Kegagalan

Pemeriksaan Gagal	Penanganan
Software tidak ditemukan	Ulangi langkah instalasi terkait. Buka terminal baru agar perubahan `PATH` berlaku
Docker Engine tidak ditemukan	Jalankan Docker Desktop dan tunggu hingga statusnya *Engine running*. Di WSL2, periksa *WSL integration* untuk Ubuntu-24.04
Memori untuk Docker kurang dari 8 GB	macOS: *Settings* → *Resources* → *Memory*. Windows: nilai `memory` di `.wslconfig`, lalu `wsl --shutdown`
Go module proxy, Docker Hub, GitHub tidak terjangkau	Akses diblokir *proxy* atau *firewall* kantor. Atur *proxy* (bagian 3) atau minta tim jaringan membuka alamat tersebut
Git server training (SSH port 22)	Port 22 keluar diblokir *firewall* kantor. Minta tim jaringan membuka port 22 ke `git.training.artivisi.id`
`docker pull`	Periksa akses Docker Hub dan `ghcr.io`, serta ruang disk

`k3d cluster create`	Pastikan Docker berjalan dan memori Docker minimal 8 GB. Jalankan ulang skrip. Percobaan kedua tidak mengulang *download* image